



Fundamentals of Database Systems

Chapter 6: SQL Language

Compiled by: Diriba G. (MSc. in CSE)

Outline

- ❖ Introduction to SQL
- ❖ DDL (Data Definition Language)
- ❖ DML (Data Manipulation Language)
- ❖ DCL (Data Control Language)

Introduction

❖ **Structured Query Language (SQL)** is a relational database language.

Amongst other things the language consists of:

- ❖ select,
- ❖ insert,
- ❖ update,
- ❖ delete,
- ❖ query and protect data.

Cont..

- ❖ SQL allows users to access data in relational DBMS such as : mySQL, Oracle, Sybase, Informix, DB2, Microsoft SQL Server, Access and others, by allowing users to describe the data the user wishes to see.
- ❖ Additionally SQL also allows users to define the data in a database and manipulate that data, for example update the data.
- ❖ We can use SQL to tell what data to retrieve or modify without telling it how to do its job.

Cont..

- ❖ At the highest level, SQL statements can be broadly categorized as follows into three types:
 - ❖ ***Data Definition Language (DDL)***, which defines the structure of the data,
 - ❖ ***Data Manipulation Language (DML)***, which retrieves or modifies data
 - ❖ ***Data Control Language (DCL)***, which defines the privileges granted to database users.

Data Definition Language (DDL)

- ❑ DDL is a standard subset of SQL that is used to define tables (database structure), and other metadata related things. The few basic commands include:
 - ❖ CREATE DATABASE,
 - ❖ CREATE TABLE,
 - ❖ DROP TABLE, and
 - ❖ ALTER TABLE.
- ❑ There are many other statements, but those are the ones most commonly used.

❖ **CREATE DATABASE**

- ❖ Many database servers allow for the presence of many databases. In order to create a database, a relatively standard command 'CREATE DATABASE' is used.
- ❖ The general format of the command is:

CREATE DATABASE <database-name>;

- ❖ The name can be pretty much anything; usually it shouldn't have *spaces* . Some databases allow hyphens, and/or underscores in the name.
- ❖ The name is usually limited in size (some databases limit the name to 8 characters, others to 32)

❖ ***DROP DATABASE***

- ❖ Just like there is a 'create database' there is also a 'drop database', which simply removes the database.
- ❖ Note that it doesn't ask you for confirmation, and once you remove a database, it is gone forever
- ❖ *DROP DATABASE <database-name> ;*

❖ *CREATE TABLE*

- ❖ Probably the most common DDL statement is 'CREATE TABLE'. Intuitively enough, it is used to create tables. The general format is something along the lines of:
- ❖ `CREATE TABLE <table-name> ( . . . . . );`
- ❖ The is where column definitions go. The general format for a column definition is the
- ❖ Column name followed by column type. For example:
- ❖ `PERSONID INT`

Cont...

- ❖ Which defines a column name personid, of type int. column names have to be comma separated, ie:

```
Create table person (  
    personid int,  
    Lname varchar(20),  
    Fname varchar(20) not null,  
    Dob date,  
    Primary key(personid));
```

Cont...

- ❖ The above creates a table named person, with person id, last name, first name, and date of birth. There is also the 'primary key' definition.
- ❖ A primary key is a column value that uniquely identifies a database record.
- ❖ So for example, we can have two 'person' records with the same last name and first name, but with different ids.

❖ ***DROP TABLE***

❖ Just like there is a ‘create table’ there is also a ‘drop table’, which simply removes the table.

❖ Note that it doesn’t ask you for confirmation, and once you remove a table, it is gone forever.

❖ *DROP TABLE <table-name> ;*

❖ ***ALTER TABLE***

- ❖ There is a command to ‘alter’ tables after you create them. This is usually only useful if the table already has data, and you don’t want to drop it and recreate it (which is generally much simpler).
- ❖ For example, Oracle allows you do add a column, but not remove a column.
- ❖ The general syntax to add a field is:

ALTER TABLE <table-name>ADD <field-name> <data-type>

Cont..

E.g. Alter table Student add (address char);

Alter table Student add (father-name varchar (60), mother-name
varchar (60), dob date); //for multiple

❖ The field declaration is pretty much exactly what it is in the ‘create table’ statement. The general syntax to drop a field is:

❖ ALTER TABLE <table-name> DROP <field-name>

❖ E.g. Alter table Student drop address;

Cont..

- ❖ Note that very few databases let you drop a field. The drop command is mostly present to allow for dropping of constraints (such as indexes, etc.) on the table.
- ❖ The general syntax to modify a field (change its type, etc.) is:
- ❖ `ALTER TABLE <table-name>`
- ❖ `MODIFY <field-name> <new-field-declaration>`

Data Manipulation Language

- ❖ This is a standard subset of SQL that is used for data manipulation. Intuitively, we need to first insert data into the database. Once it's there, we can retrieve it, modify it, and delete it. These directly correspond to:
 - ❖ INSERT,
 - ❖ SELECT,
 - ❖ UPDATE, and
 - ❖ DELETE statements.

❖ ***THE INSERT STATEMENT***

❖ To get data into a database, we need to use the 'insert' statement. The general syntax is:

❖ **INSERT INTO <table-name> (<column1>,<column2>,<column3>,...)**
❖ **VALUES (<column-value1>,<column-value2>,<column-value3>);**

❖ The column names (i.e.: column1, etc.) must correspond to column values (i.e.: column-value1, etc.).

❖ There is a short-hand for the statement:

❖ **INSERT INTO <table-name>**
❖ **VALUES (<column-value1>,<column-value2>,<column-value3>);**

Cont..

- ❖ A typical example, of inserting the 'person' record we've created earlier would be:
- ❖ *insert into person (personid, lname, fname, dob)
values(1, 'doe', 'john', '1956-11-23');*

❖ ***THE SELECT STATEMENT***

- ❖ Probably the most used statement in all of SQL is the SELECT statement. The select statement has the general format of:

❖ SELECT <column-list>

FROM <table-list>

WHERE <search-condition>

Cont..

❖ *Example1.* We want to get all columns and rows from the table cars.

Select car_number, make, model, year, price

❖ FROM cars;

❖ To display the whole table

❖ Select *

❖ from cars;

❖ **Example 2.** We select from table Employee the Last_Name and Car_Number when the First_Name is Sean

```
❖ SELECT Last_Name, Car_Number
❖ FROM Employees
❖ WHERE First_Name = 'Sean';
```

❖ **Example 3.** The command

```
❖ SELECT Make
❖ FROM Cars
❖ WHERE Price > 60000;
```

❖ *Example 4.* The command

```
❖ SELECT First_Name, Last_Name  
❖ FROM Employees  
❖ WHERE Car_Number IS NOT NULL;
```

❖ *Example 5.* An application of the arithmetic operation + is as follows

```
❖ SELECT Model, Price + 5000  
❖ FROM Cars;
```

Cont..

❖ *Example 6.* An application of the logical AND is

❖ **SELECT First_Name, Last_Name**
❖ **FROM Employees**
❖ **WHERE Employee_Number < 10010 AND**
Car_Number IS NULL;

❖ *Example 7.* The query

❖ **SELECT COUNT(*)**
❖ **FROM Employees**
❖ **WHERE Car_Number IS NULL;**

Cont..

❖ **Example 8.** To sort data we use the ORDER BY clause. It sorts rows on the basis of columns. The ORDER BY clause is optional. By default the database system orders the rows in ascending order.

```
❖ SELECT Last_Name, First_Name, Date_of_Birth
❖ FROM Employees
❖ ORDER BY Last_Name, First_Name;
```

❖ *THE WHERE CLAUSE*

- ❖ The WHERE clause is used in UPDATE, DELETE, and SELECT statements, and has the same format in all these cases.
- ❖ It has to be evaluated to either true or false. Table 1 lists some of the common operators.

=	equals to
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
<>	not equal to

Table 1: SQL Operators

❖ **Like clause**

- ❖ **Like** clause is used as condition in SQL query. **Like** clause compares data with an expression using wildcard operators. It is used to find similar data from the table.

Wildcard operators

- ❖ There are two wildcard operators that are used in like clause.
- ❖ ☐ **Percent sign %**: represents zero, one or more than one character.
- ❖ ☐ **Underscore sign _**: represents only one character.

Examples

- ❖ `SELECT * from Student where s_name like 'A%';`
- ❖ The above query will return all records where **s_name** starts with character 'A'.

- ❖ `SELECT * from Student where s_name like '_d%';`
- ❖ The above query will return all records from **Student** table where **s_name** contain 'd' as second character.

Order By Clause

❖ **Syntax:** SELECT column-list|* from table_name **order by** asc|desc;

Eg. SELECT * from Emp **order by** salary ASC;

❖ The above query will return result in ascending order of the **salary**.

SELECT * from Emp order by salary DESC;

❖ ***THE UPDATE STATEMENT***

- ❖ The update statement is used for changing records. The general syntax is:

```
UPDATE <table-name> SET <column1> = <value1>, <column2> = <value2>, ...  
WHERE <criteria>
```

- ❖ An example of the use would be

```
❖ UPDATE PERSON  
    SET FNAME='Clark', LNAME='Kent'  
    WHERE FNAME='Superman';
```

❖ ***THE DELETE STATEMENT***

❖ The 'delete' is used to remove elements from the database. The syntax is very similar to update and select statements:

❖ **DELETE FROM <table-name>**

❖ **WHERE <criteria>**

❖ An example of the use would be

❖ **DELETE FROM PERSON**

❖ **WHERE PERSONID=12345;**

Data Control Language

- ❖ This is a standard subset of SQL that is used for security management.

Most databases have their own flavored syntax, but generally, there exist two commands:

- ❖ GRANT, and

- ❖ REVOKE,

- ❖ these ‘grant’ and ‘remove’ privileges.

GRANT Statement

- ❖ The general format is something like this:
 - ❖ GRANT <privilege> ON <object> TO <who> ;
 - ❖ eg. Grant update on employee to Obse;

REVOKE Statement

- ❖ The revoke is the opposite of GRANT. The general format is:
 - ❖ REVOKE <privilege> ON <object> FROM <who> ;
 - ❖ eg. Revoke update on employee from Obse;

End of Chapter 6

Thank You!